Phone Home

June 19, 1976: Viking 1 enters orbit around Mars. Every reading it takes is worthless unless it comes *back* to Earth.

Marcus Birkenkrahe

COSC 1210 · CS I · Session 1 of 3 · Augustana University

Fall 2026 · Downey, *Think Python* 3e, ch. 6

Mission plan: three sessions, from orbit to the surface

Session 1 · today: Phone home

`return`, `None`, pure functions, `assert`

Session 2: Where to land

Every path returns, Boolean functions, incremental development

Session 3: Talking through the orbiter

Recursion that returns a value

Last week every function **printed**. This week they hand their answers **back**.

Review · countdown: three quick-fire questions

1. What stops a recursive function?
   → The base case: a branch that does not call the function again.

2. What happens when you call `countdown(7)`?

```
def countdown(n):
    if n == 0:
        print("Liftoff!")
    else:
        print(n)
        countdown(n - 2)
```

   → 7, 5, 3, 1, −1, . . . never meets 0: infinite recursion, `RecursionError`. Fix: `n <= 0`.

3. A function prints `Fuel: NO-GO`. Can the code that called it use that verdict to stop the launch?
   → No – printed text goes to the screen, not back to the caller. The caller has nothing to test. That is today's problem.

Ask go_nogo for its verdict

→ **Switch to editor · code-along 1 · TRY: predict**

```
def go_nogo(system, reading, minimum):
    if reading >= minimum:
        print(f"{system}: GO")
    else:
        print(f"{system}: NO-GO")

verdict = go_nogo("Fuel", 80, 90)
print("mission received:", verdict)
```

Predict: what does `verdict` hold – before you run it?

What mission control gets

A function without `return` hands back `None`

```
Fuel: NO-GO
mission received: None
```

The verdict appears on the screen – and is gone. **None** means “no value”. You have seen it before: `print` itself returns `None`.

Hand the verdict back

→ Switch to editor · code-along 2 · TRY (2 min)

1. Copy go_nogo into a new function `status(reading, minimum)`
2. Change it so the caller gets `"GO"` or `"NO-GO"` **back** – no printed line
3. Run the three lines below

```
verdict = status(80, 90)
print("mission received:", verdict)
if verdict == "NO-GO":
    print("HOLD: fuel")
```

Expected

```
mission received: NO-GO
HOLD: fuel
```

print or return

`return` ends the function and hands a value back

Void function: `print`

Shows the value on the screen. The caller gets **None**.

Fruitful function: `return`

The caller gets the value – to store, print, test, or pass on.

```
status(96, 90)          # returned - and dropped
print(status(96, 90))   # returned - and printed: GO
```

Celsius in, Kelvin back

→ **Switch to editor · code-along 3 · TRY (3 min)**

1. Write `to_kelvin(celsius)`: $K = °C + 273.15$
2. Store the Kelvin value of the pre-dawn reading, $-86\,°C$, in `dawn_k` and print it
3. Print the difference between $-33\,°C$ and `dawn_k`, in Kelvin

Expected

```
187.14999999999998
53.0
```

Returned values go anywhere

```
print(round(to_kelvin(-86), 2))     # 187.15
print(status(to_kelvin(-33), 200))  # GO
```

Viking 1, on the surface

−86 °C

before dawn

−33 °C

in the afternoon

A daily swing of **53 degrees** – the same in Celsius and Kelvin. Only the zero point differs.

Source: NASA, Viking 1 mission page

Question · predict first: what does this print?

```
def half(x):
    print(x / 2)

def half2(x):
    return x / 2

a = half(10)
b = half2(10)
print(a, b)
```

Answer

```
5.0         # printed inside half(10)
None 5.0    # print(a, b)
```

`half(10)` prints `5.0` and returns `None`.
`half2(10)` prints nothing and returns `5.0`.

Pure functions: same input, same output – and nothing else

`to_kelvin` and `status` are **pure**: no side effects – no printing, no input, no changes outside. They only return a value.

They compose

One value flows straight into the next function.

They are predictable

Same input, same output – every time.

They separate jobs

The function computes; the caller prints, stores, or transmits.

They are testable

From Earth, with one line per test: `assert`.

A test from Earth finds a bug

→ Switch to editor · code-along 4 · TRY (3 min)

```
def to_fahrenheit(celsius):
    return celsius * 5 / 9 + 32

assert to_fahrenheit(-40) == -40, f"expected -40, got {to_fahrenheit(-40)}"
```

1. Run it and read the message
2. Find the bug and fix the formula
3. Add a second test: 100 °C is 212 °F

One-line tests: the message points at the bug

```
AssertionError: expected -40, got 9.777777777777779
```

`5 / 9` is upside down. With `celsius * 9 / 5 + 32` both tests pass – silently.

You can only test what a function returns

A printing function returns `None` – and `None == -40` is `False`.

Build the message, don't print it

→ Switch to editor · code-along 5 · TRY (3 min)

Write `downlink(sensor, value, unit)` that **returns** one line for Earth. The caller decides when to send it.

```
message = downlink("temperature",
    round(to_kelvin(-86), 2), "K")
print(message)
print(downlink("pressure", 7.6, "mbar"))
```

Expected

```
V01 temperature: 187.15 K
V01 pressure: 7.6 mbar
```

The caller decides: same function, input from the keyboard

```
sensor = input("Sensor: ")
value = float(input("Value: "))
unit = input("Unit: ")
print(downlink(sensor, value, unit))
```

```
Sensor: wind
Value: 7.2
Unit: m/s
VO1 wind: 7.2 m/s
```

`downlink` did not change at all. The keyboard only changed what the caller passed in – that is the payoff of returning instead of printing.

Practice: weather station (15 min)

→ **Switch to editor · write the lander's conversion software**

1. `to_celsius(fahrenheit)` returns $(F - 32) \cdot 5 / 9$. Test with `assert`: $-40\,°F$ is $-40\,°C$, $212\,°F$ is $100\,°C$
2. `to_kpa(mbar)` returns kilopascal (1 kPa = 10 mbar). Test: 7.6 mbar is 0.76 kPa
3. `weather_report(fahrenheit, mbar)` *returns* one string, using both functions; round to whole degrees and two decimals
4. Why could you not build `weather_report` from last week's printing `to_celsius`?

```
print(weather_report(-123, 7.6))
```

```
VL1 weather: -86 C, 0.76 kPa
```

Checkout

What is the difference between `print` and `return` in a function?

And what does the caller get in each case?

→ `print` shows a value on the screen; the caller gets `None`.

→ `return` hands the value back – to store, print, test, or pass on.

Summary

- A function without **return** returns **None**. Printed output is gone – the caller cannot use it
- **return** ends the function and hands a value back: store it, print it, use it in an expression, or pass it on
- **Fruitful** functions return a value; **void** functions only do something, like printing
- **Pure** functions return a value and have no side effects: they compose, they are predictable, they are easy to test
- `assert condition, message` is a one-line test: silent if True, `AssertionError` with your message if False
- Build a message with `return` and let the caller decide when to send it

Glossary: terms from session 1

#	Term	Meaning
1	`return`	Ends a function and hands a value back
2	return value	The value a function hands to its caller
3	fruitful function	A function that returns a value
4	void function	A function that only does something; `None`
5	pure function	Returns a value, no side effects
6	side effect	Any effect besides returning, e.g. printing
7	`assert`	One-line test: stops if its test is False
8	`AssertionError`	The error a failing `assert` raises

Next session: Where to land

The orbiter's photos show that the planned landing site is rougher than expected. The landing is postponed.

- Is a site safe – **True** or **False**?
- And with a returned verdict, `mission` can finally stop the countdown